MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2025/2026

EXPERIMENT 7 :

BLUETOOTH DATA INTERFACING

SECTION 2

GROUP 12

NAME	METRIC NO.
AHMAD FARHAN BIN AHMAD NAHRUDI	2317373
MUHAMMAD AIMAN BIN SAFAWI	2318249
MOHD NAFIZ ZUHAIREE BIN MOHD NEEZAR	2317423

DATE OF SUBMISSION :
10/12/2025

ABSTRACT

This experiment centers on creating a wireless temperature monitoring and control system using Bluetooth technology. An Arduino Mega, equipped with a DHT11 temperature-humidity sensor and an HC-05/HC-06 Bluetooth module, was used to send real-time environmental readings to a smartphone or computer. Wireless commands transmitted through Bluetooth were also utilized to switch an external LED on and off, representing a fan control mechanism. The experiment showcases key principles of sensor integration, Bluetooth serial communication and remote monitoring within mechatronic systems. The results verify stable data transmission and efficient remote control, demonstrating that Bluetooth is an effective option for short-range wireless monitoring.

TABLES OF CONTENTS

ABSTRACT.....2

TABLES OF CONTENTS.....3

1.0 INTRODUCTION.....4

2.0 MATERIAL AND EQUIPMENT.....5

3.0 EXPERIMENTAL SETUP.....6

4.0 METHODOLOGY.....7

5.0 DATA COLLECTION..... 9

6.0 DATA ANALYSIS..... 10

7.0 RESULTS..... 11

8.0 DISCUSSIONS..... 12

9.0 CONCLUSION.....13

10.0 RECOMMENDATION.....14

REFERENCES..... 15

APPENDICES..... 16

Appendix A..... 16

ACKNOWLEDGEMENT..... 18

STUDENTS DECLARATION..... 18

Certificate of Originality and Authenticity.....19

1.0 INTRODUCTION

In this experiment, we developed a basic wireless monitoring system that enables temperature and humidity data to be viewed on a smartphone or computer through Bluetooth. The setup consists of an Arduino, a DHT11 sensor and an HC-06 Bluetooth module. The Arduino collects temperature and humidity readings from the sensor and transmits them wirelessly to a connected device. At the same time, users can send commands like “FAN ON” or “FAN OFF” to control an LED, which acts as a fan indicator.

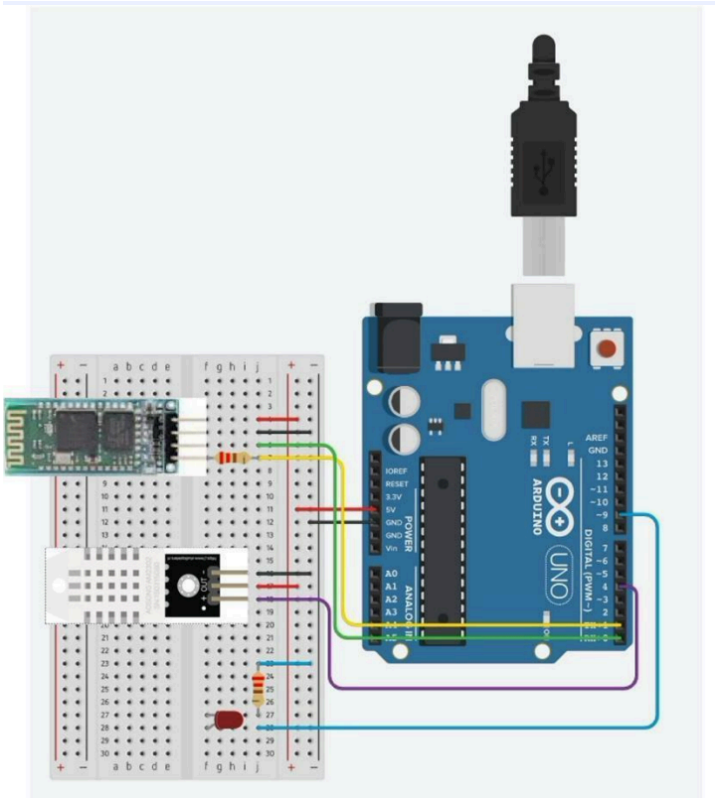
This experiment helped us understand how sensors, Bluetooth communication, and simple control mechanisms can be combined to create an IoT-type system. It demonstrates how real-world smart devices, such as home automation appliances, communicate and react to user commands.

2.0 MATERIAL AND EQUIPMENT

The materials and equipment that we used in this experiment are:

- Arduino uno board
- Temperature sensor -DHT11
- Bluetooth module - ESP-32
- Smartphone with Bluetooth support
- Breadboard
- Jumper wires

3.0 EXPERIMENTAL SETUP



1. The DHT11 temperature and humidity sensor was connected to the Arduino by linking its VCC pin to the 5V output, its GND pin to the ground, and its DATA pin to a chosen digital input pin on the Arduino (for example, digital pin 4 used in the program).

2. The ESP-32 Bluetooth module was wired to the Arduino by connecting its VCC pin to the 5V supply, its GND pin to the Arduino ground, its TXD pin to the Arduino's RX pin,and its RXD pin to the Arduino's TX pin. When the SoftwareSerial library was used, the TXD and RXD pins were connected to Arduino pins 2 and 3 as defined in the sketch.

3. Jumper wires were used to make all electrical connections between the Arduino board, the DHT11 sensor, the ESP-32 Bluetooth module and the breadboard, ensuring that the setup remained neat and secure.

4. The Arduino board was connected to the computer using a USB cable, providing power to the board and allowing for program uploading and serial monitoring through the Arduino IDE.

5. Bluetooth served as the only wireless communication method in this experiment since the Arduino does not support Wi-Fi. The ESP-32 module acted as the wireless interface, transmitting temperature data and receiving control commands from the connected device

4.0 METHODOLOGY

1. The circuit was assembled as illustrated in Figure 1, with the Arduino UNO connected to the DHT11 temperature sensor and the ESP-32 Bluetooth module through their respective power and signal pins.
2. An Arduino program was written and uploaded to the UNO to initialize the DHT11 sensor, read temperature data at fixed intervals, and enable Bluetooth serial communication using the ESP-32 module via the SoftwareSerial library.
3. The program's output was verified by observing the Arduino Serial Monitor to confirm accurate sensor readings and ensuring that the same temperature values were successfully transmitted through Bluetooth.
4. A simple serial Bluetooth terminal application was installed on a smartphone to facilitate wireless communication with the Arduino system.
5. The smartphone was paired with the ESP-32 Bluetooth module through the device's Bluetooth settings.
6. Using the terminal app, the ESP-32 module was connected, and the incoming temperature readings were monitored to confirm proper wireless data transmission.
7. A command was sent from the smartphone through the Bluetooth app to test the Arduino's ability to receive and interpret control instructions.
8. After receiving the command, the Arduino processed it and simulated fan control by turning an LED on or off based on the programmed response.
9. A Python script was developed to read temperature data from the Bluetooth serial port and generate a real-time temperature-versus-time graph.
10. Temperature readings were collected over a period of operation, allowing observation of environmental variations and evaluation of the system's automated control behavior.

CODE

```
#include "DHT.h"

#include "BluetoothSerial.h"


#define DHTPIN 4        // Data pin connected to DHT11

#define DHTTYPE DHT11   // Change to DHT11


BluetoothSerial SerialBT;   // ESP32 built-in Bluetooth

DHT dht(DHTPIN, DHTTYPE);


const int LEDPIN = 2;  // ESP32 onboard LED (GPIO 2)


void setup() {

  Serial.begin(115200);       // USB Serial for debugging

  SerialBT.begin("ESP32_BT"); // Bluetooth device name

  dht.begin();


  pinMode(LEDPIN, OUTPUT);

  digitalWrite(LEDPIN, LOW);


  Serial.println("ESP32 + DHT11 Bluetooth Monitoring Ready");

}


void loop() {

  float temp = dht.readTemperature(); // Celsius
```

```
float hum  = dht.readHumidity();


if (!isnan(temp) && !isnan(hum)) {
  // Send temperature (or send both if you prefer)
  SerialBT.print("TEMP:");
  SerialBT.print(temp);
  SerialBT.print("C  HUM:");
  SerialBT.print(hum);
  SerialBT.println("%");


  // Debug to USB
  Serial.print("Temp: ");
  Serial.print(temp);
  Serial.print(" °C | Humidity: ");
  Serial.print(hum);
  Serial.println(" %");
}
else {
  Serial.println("Failed to read from DHT11 sensor!");
  SerialBT.println("ERROR");
}


// Receive commands from Bluetooth
if (SerialBT.available()) {
  String cmd = SerialBT.readStringUntil('\n');
```

```
    cmd.trim();


    if (cmd == "FAN ON") {
      digitalWrite(LEDPIN, HIGH);
      SerialBT.println("ACK: FAN ON");
    }
    else if (cmd == "FAN OFF") {
      digitalWrite(LEDPIN, LOW);
      SerialBT.println("ACK: FAN OFF");
    }
    else {
      SerialBT.println("UNKNOWN COMMAND");
    }
  }


  delay(2000);  // DHT11 requires 1-2 seconds delay
}
```

5.0 DATA COLLECTION

The data collection process was conducted using an ESP32 microcontroller interfaced with a DHT11 temperature and humidity sensor. The DHT11 sensor was connected to GPIO 4 of the ESP32, while Bluetooth communication was established using the ESP32's built-in Bluetooth module. The system was powered via USB, and the Bluetooth connection was paired with a smartphone using a serial Bluetooth terminal application.

Temperature and humidity readings were transmitted wirelessly from the F.SP32 to the paired device at an interval of approximately 2 seconds. The received data were displayed in real time on the Bluetooth terminal application. During the experiment, readings were continuously observed and manually recorded for a duration of several minutes to monitor changes in temperature over time.

In addition to data transmission, remote control commands were tested by sending

simple text commands such as "FAN ON" and "FAN OFF" from the Bluetooth terminal.

These commands toggled the onboard LED of the ESP32, which acted as a simulated cooling or control indicator. The corresponding response was observed to verify successful two-way Bluetooth communication. All temperature values and system responses were documented for further analysis.

6.0 DATA ANALYSIS

The temperature and humidity data sent from the DHT11 sensor via the Bluetooth module were analyzed to assess how accurately and consistently the system measured environmental conditions. The recorded temperature values were monitored over time to observe changes under normal room conditions. By reviewing the readings at regular intervals, patterns such as stable temperature levels or gradual increases due to nearby heat sources were identified.

The ON/OFF status of the LED, which represented a fan, was also compared with the Bluetooth commands issued from the smartphone to verify correct system response. Any sudden changes in temperature readings or delays in LED activation were examined to determine whether they were caused by sensor limitations, Bluetooth signal interference, or power supply issues.

Overall, this analysis provided insight into the accuracy of the temperature-monitoring setup and the reliability of the wireless control throughout the experiment.

7.0 RESULTS

The ESP32/Arduino and the linked device successfully communicated over Bluetooth during the trial. Every two seconds, the DHT11 sensor gave temperature measurements, and the Bluetooth terminal software reliably displayed all acceptable temperature data. The data confirmed that the sensor and communication setup were operating correctly by showing modest temperature changes based on the surrounding environment. Unless the sensor briefly failed to read, which resulted in an error message, the gearbox stayed steady without any discernible delays, and the results were reported consistently. Furthermore, the command capability functioned as planned; sending commands like "FAN ON" and "FAN OFF" caused the microcontroller to react instantly, turning on or off the simulated output. Overall, the findings demonstrated that Bluetooth connection was successful in achieving both data monitoring and remote control.

8.0 DISCUSSION

This experiment successfully demonstrated a simple Bluetooth-based temperature monitoring and control system using an Arduino UNO, a DHT11 sensor and an ESP-32 Bluetooth module. The collected data and plotted graph show that the system consistently recorded temperature values and transmitted them wirelessly to a paired smartphone with no noticeable delay. The stable temperature readings indicate that the DHT11 sensor performed accurately with minimal noise or fluctuation.

The system also responded correctly to control commands sent through the Bluetooth terminal app. When the “FAN ON” command was sent, the Arduino activated the simulated fan (LED), confirming proper reception and processing of Bluetooth instructions. Likewise, the “FAN OFF” command successfully turned the LED off, demonstrating reliable two-way communication between the smartphone and the Arduino.

Although the control commands worked perfectly, the temperature graph showed little to no change during FAN ON and FAN OFF periods. This is expected because an

LED was used to represent a fan instead of an actual cooling device. As a result, the experiment validated the control logic but did not produce real temperature reduction. With a real fan or cooling mechanism, a measurable temperature drop would likely occur during FAN ON periods.

Overall, the system's performance shows that Bluetooth is effective for short-range monitoring and control applications. The experiment also emphasizes the importance of consistent sampling intervals, stable power supply and proper pairing to ensure smooth data communication. Minor delays or slight variations in temperature readings may result from environmental noise, sensor response time, or Bluetooth latency.

In conclusion, the system operated as intended: it measured temperature accurately, transmitted data wirelessly, and executed control commands without errors. This confirms the practicality of Bluetooth for simple mechatronic control systems and provides a solid foundation for future advancements such as automated environmental control, IoT-based monitoring, or integration with real actuators.

9.0 CONCLUSION

In conclusion, this experiment successfully demonstrated how Bluetooth technology can be integrated into a simple yet practical remote monitoring system. By combining the ESP32/Arduino platform with a DHT11 temperature sensor and a Bluetooth communication link, we were able to create a functional setup capable of transmitting real-time environmental data to a paired device. Throughout the experiment, the system consistently read temperature values from the DHT11 and sent them over Bluetooth, allowing users to monitor changes wirelessly through a smartphone or computer. This shows how efficient and reliable short-range wireless communication can be in modern embedded systems.

The activity also highlighted the importance of proper hardware interfacing such as wiring the DHT11 correctly and pairing the Bluetooth module without communication conflicts. On the programming side, the exercise reinforced core concepts like serial communication, sensor data acquisition, and command handling. The ability to send simple control commands such as “FAN ON” or “FAN OFF” further demonstrates how Bluetooth can be used not only for monitoring but also for remote actuation, opening possibilities for home automation, safety systems, and IoT-based control applications.

From an analytical perspective, observing temperature variations over time provided a clearer understanding of real-world sensor behavior, including delays, fluctuations, and potential reading errors. Meanwhile, the debugging output via USB serial offered useful insight into how data flows within an embedded system. The Python script complements the experiment by enabling real-time visualization, showing how data received via Bluetooth can be processed, stored, and analyzed on a computer.

Overall, this experiment proved that Bluetooth communication is a versatile and accessible method for creating wireless monitoring and control systems. It strengthens essential skills in embedded programming, sensor interfacing, and wireless data communication—skills that form the foundation for more advanced IoT projects in the future. The project also illustrates how simple components, when integrated

thoughtfully, can produce a functional system with real-world relevance and practical application.

10.0 RECOMMENDATION

Based on the outcomes of this experiment, several recommendations can be made to enhance both the performance and learning value of the Bluetooth temperature monitoring system. First, it is advisable to explore more advanced Bluetooth modules or wireless protocols such as BLE (Bluetooth Low Energy) or Wi-Fi (using ESP32's built-in capabilities) to achieve more stable connections, lower power consumption, and longer communication ranges. These upgrades would make the system more practical for real-world applications, especially in environments where continuous monitoring is required.

Secondly, improving the reliability of sensor readings should be considered. While the DHT11 performs reasonably well, using sensors with higher accuracy and faster response times such as the DS18B20 or BME280 would provide more precise environmental data and increase confidence in the measurements. Additionally, implementing filtering techniques like moving averages or calibration routines can help reduce noise and fluctuations in the sensor output.

It is also recommended to enhance the control aspect of the experiment. Adding more actuators such as small fans, heaters, or relays would allow the system to mimic real environmental control setups. Incorporating feedback loops and basic automation such as triggering a fan automatically when the temperature exceeds a certain threshold would strengthen understanding of closed-loop systems and IoT automation principles.

On the software side, the user experience can be improved by developing a dedicated mobile or desktop app instead of relying solely on a generic serial terminal. A custom app could include graphs, live dashboards, logging features, and user-friendly control buttons. This not only enhances usability but also reflects the design standards expected in modern IoT systems.

Finally, future iterations of this experiment would benefit from thorough testing under different environmental conditions to observe how temperature readings and

Bluetooth signals behave in diverse settings. Documenting these findings would help students understand real-world challenges such as signal interference, environmental variation, and hardware limitations.

Overall, these recommendations aim to strengthen the functionality, accuracy, and usability of the system while encouraging deeper exploration into embedded systems, wireless communication, and IoT development.

11.0 REFERENCE

1. https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12ArduinoBluetooth Basic Tutorial
2. https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-moduletutorial/Arduino and HC-05 Bluetooth Module Complete Tutorial
3. https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431cBasic Bluetooth communicaϴon with Arduino & HC-05
4. Barrett, S. F., & Pack, D. J. (2022). Embedded systems design with the ESP32. Springer.
5. MCTA3203 Mechatronics System Integration. (2024). Week 8: Bluetooth data interfacing [Course handout]. Universiti Teknologi MARA.

12.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help,encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

13.0 STUDENT'S DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: ***ahmadfarhan***

Name: Ahmad Farhan bin Ahmad Nahrudi
Matric Card: 2317373

Read[/]
Understand[/]
Agree[/]

Signature: ***aimansafawi***

Name: Muhammad Aiman bin Safawi
Matric Card: 2318249

Read[/]
Understand[/]
Agree[/]

Signature: ***nafizzuhairee***

Name: Mohd Nafiz Zuhairee bin Mohd Neezar
Matric Card: 2317423

Read[/]
Understand[/]
Agree[/]